

Building agentic workflows from scratch

As mentioned, `AgentWorkflow` is a pre-built `Workflow`, so you've already encountered many of the concepts you need to build a workflow from scratch.

You can easily build your own workflows directly to create multi-agent systems that are customized to your needs. There's a number of reasons you might want to branch out from `AgentWorkflow` to rolling your own workflow:

-  **Fine-grained flow control:** Workflows allow looping, branching, parallel calls and map-reduce behavior. Workflows allow you to precisely specify how data gets passed around your system.
-  **Structured inputs and outputs:** If you want your inputs and outputs to be richer than simple strings, or to kick off with more than just a `user_msg`, workflows allow you to do that.
-  **Multimodality:** LlamaIndex can handle more than just text! You can handle images, audio and video as well.
-  **Query planning:** a custom workflow can execute complex planning of how to run based on its inputs, rather than immediately jumping into a team.
-  **Reflection:** A powerful technique for agents is their ability to examine their own output and decide whether it's sufficient, or if they need to try again. Using looping, you can implement this pattern.

Creating a new workflow

Under the hood, Workflows are regular Python classes. They are defined as a series of `steps`, each of which receives certain classes of events and emits certain classes of events.

Here's the most basic form of a workflow, with a single step:

! Note: Run the hidden cells below before running the rest of the code. !

1 hidden cell

```
# Fix: Install the llama-index package before importing
!pip install llama-index

from llama_index.core.workflow import (
    StartEvent,
    StopEvent,
    Workflow,
    step,
)

class MyWorkflow(Workflow):
    @step
    async def my_step(self, ev: StartEvent) -> StopEvent:
        # Do something here
        return StopEvent(result="Hello, world!")
```

```
Requirement already satisfied: llama-index in /home/repl/.venv-
user/lib/python3.12/site-packages (0.14.24)
Requirement already satisfied: llama-index-core<0.15.0,>=0.14.24 in
/home/repl/.venv-user/lib/python3.12/site-packages (from llama-index) (0.14.24)
Requirement already satisfied: llama-index-embeddings-openai<0.7,>=0.6.0 in
/home/repl/.venv-user/lib/python3.12/site-packages (from llama-index) (0.6.0)
Requirement already satisfied: llama-index-llms-openai<0.8,>=0.7.0 in
/home/repl/.venv-user/lib/python3.12/site-packages (from llama-index) (0.7.10)
Requirement already satisfied: nltk>=3.9.3 in /home/repl/.venv-
user/lib/python3.12/site-packages (from llama-index) (3.10.3)
Requirement already satisfied: aiohttp<4,>=3.8.6 in /home/repl/.venv-
user/lib/python3.12/site-packages (from llama-index-core<0.15.0,>=0.14.24->llama-
index) (3.12.15)
Requirement already satisfied: aiosqlite in /home/repl/.venv-
user/lib/python3.12/site-packages (from llama-index-core<0.15.0,>=0.14.24->llama-
index) (0.22.1)
Requirement already satisfied: banks<3,>=2.3.0 in /home/repl/.venv-
user/lib/python3.12/site-packages (from llama-index-core<0.15.0,>=0.14.24->llama-
index) (2.5.1)
Requirement already satisfied: dataclasses-json in /home/repl/.venv-
user/lib/python3.12/site-packages (from llama-index-core<0.15.0,>=0.14.24->llama-
index) (0.6.7)
Requirement already satisfied: deprecated>=1.2.9.3 in /home/repl/.venv-
user/lib/python3.12/site-packages (from llama-index-core<0.15.0,>=0.14.24->llama-
index) (1.2.18)
Requirement already satisfied: dirtyjson<2,>=1.0.8 in /home/repl/.venv-
user/lib/python3.12/site-packages (from llama-index-core<0.15.0,>=0.14.24->llama-
index) (1.0.8)
Requirement already satisfied: filetype<2,>=1.2.0 in /home/repl/.venv-
```

This new `MyWorkflow` class:

- Uses the `@step` decorator to declare a function to be a step
- Has a single step called `my_step` which accepts a `StartEvent`. `StartEvent` is a special event which is always generated when a workflow first runs.
- `my_step` returns a `StopEvent`, which is another special event. When a `StopEvent` is emitted the workflow returns it and stops running.

We instantiate it and run it just like you ran the `AgentWorkflow`:

```
workflow = MyWorkflow(timeout=10, verbose=False)
result = await workflow.run()
print(result)
```

Hello, world!

</> Aside: running a workflow in regular Python

Workflows are async by default, so we use `await` to get the result of the `run` command. This will work fine in a notebook environment; if you are using a vanilla Python script, you will need to import `asyncio` and wrap your code in an async function, like this:

```
async def main():
    w = MyWorkflow(timeout=10, verbose=False)
    result = await w.run()
    print(result)

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

 Copy

Since we're in a notebook right now, we won't execute the above code as it won't work!

Visualizing a workflow

A great feature of workflows is the built-in visualizer, which we will install now:

```
!pip install llama-index-utils-workflow -q -q
```

Let's visualize the simple workflow we just created:

```
from llama_index.utils.workflow import draw_all_possible_flows  
  
draw_all_possible_flows(MyWorkflow, filename="basic_workflow.html")
```

basic_workflow.html

This creates a file called `basic_workflow.html` in the notebook's file system. You can open up the file pane and download the file, then open it in your browser. You'll see something like this:

